

pi 解剖 四个工具，六个"不做" 和一个 9.9 万星的实验

A MINIMAL AGENT HARNESS · DEV NOTES

4

TOOLS

6

NO'S

98.8k

STARS*

250+

RELEASES

* 单源页面抓取（2026-08-29），未能二次核验。数据与引文的出处见文末来源页。

2025-11-30 发布



2026-01-31 Ronacher 评述



2026-04-08 加入 Earendil



2026-08-28 v0.84.4

TOC 七个问题讲清一个项目

AGENDA

篇幅 20 页 · ≈15 分钟

前半回答"它是什么、凭什么", 后半回答"代价是什么"

01 缘起：为什么又是一个 agent

P4

02 架构解剖：时序 + 五层薄切 + 数字

P5-7

03 四工具哲学 + 六个"不做"

P8-9

04 上手：十分钟到会话树

P10-11

05 扩展系统 + 完整源码

P12-13

06 横向定位：底盘 vs 整车

P14-15

07 批判：六笔账 + 三段原声 + 边界

P16-18

REF 提炼 + 参考来源

P19-20

00 开篇：三个问题

OPENING

方法 公开材料 only

2026 年的 agent 竞赛是一道加法题，pi 反着来：四个工具，六个"不做"

Q1

它是什么？

Mario Zechner (libGDX 作者) 写的极简 agent harness：默认四个工具，README 里有整节"刻意不做什么"。

Q2

凭什么？

极简内核 + 极致可扩展：sub-agent、plan mode、权限门全都有官方扩展答案，OpenClaw 在它之上长成爆款。

Q3

代价是什么？

无 MCP、无现成生态、TS 门槛、自建维护成本、商业化悬念、好评圈层——六笔账逐条算。

METHOD 全文基于公开材料 (README / 源码 / 博客 / 批评文)，引语保留英文原句，单一看源的数据当场标注。没跑过基准，没有利益相关。

01 缘起：两年三级跳，从个人项目到爆款底座

WHY

动机

反塞满 · 反黑盒

"往上下文窗口里塞了太多东西，还把正在发生的事藏起来"——两个不满，两个设计决策

2025-11-30

博文宣布 pi

标题自带 opinionated (固执己见)



2026-01-31

Ronacher 长文评述

"a minimal agent harness" (软文, 立场后述)



2026-04-08

《I've sold out》

带 pi 加入 Earendil, 仓库与 npm 包名迁移



2026-08-28

v0.84.4

8 月 5 连发 · 累计约 250 个 release

作者是谁

Mario Zechner = badlogic

libGDX 作者，开源社区养了十几年。这种履历自带信用：见过项目怎么长大，也见过它怎么死。

引爆点

OpenClaw 的底座

2026 年初走红的 OpenClaw (旁证称病毒式传播，未量化核实)，底层 agent 框架就是 pi。做底盘的项目，因别人的整车被看见。

定位自白

"...but this one is yours"

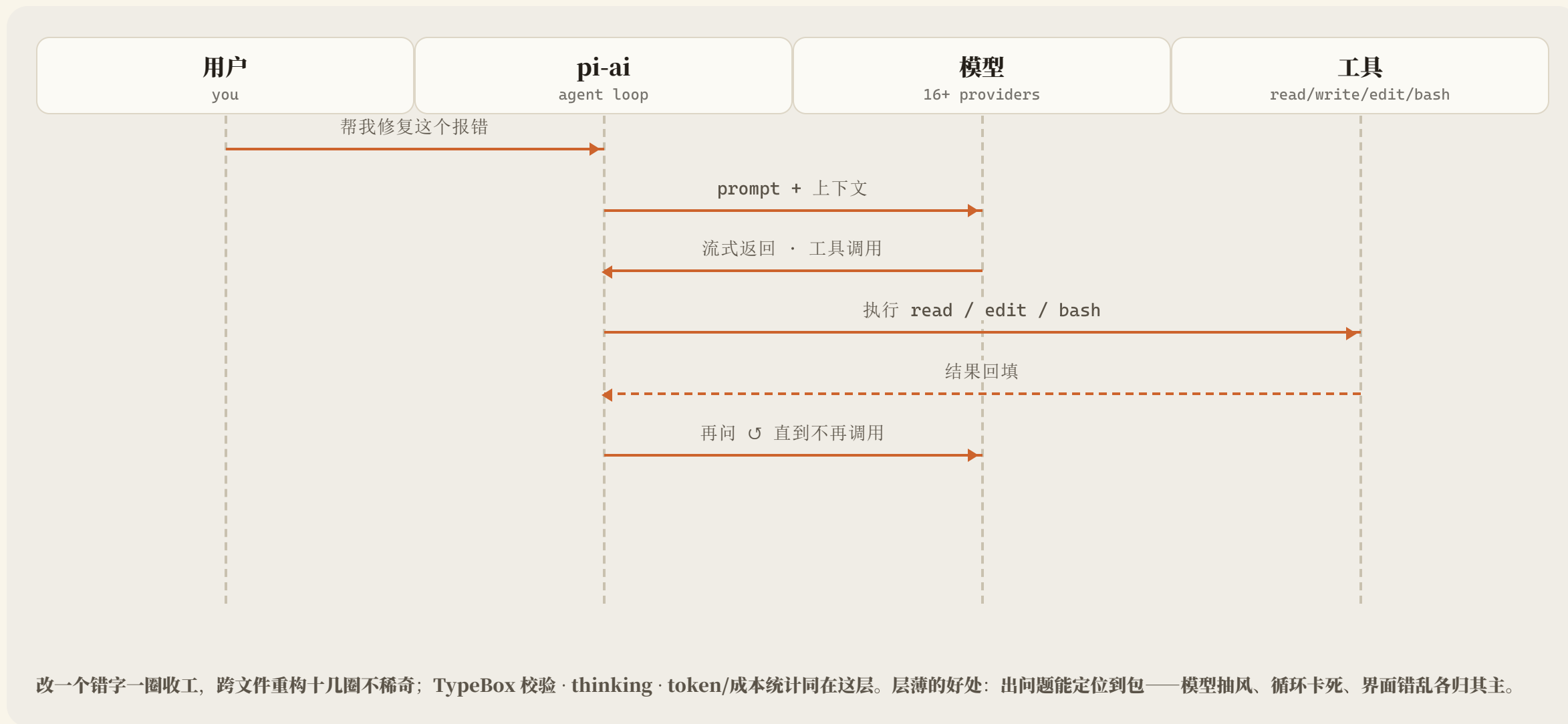
pi.dev 首页原话：agent harness 有很多，但这一个是你的。所有权，而不是体验，才是卖点。

02 架构解剖：一轮对话是一圈循环，没有步数上限

ARCH · SEQ

产出 agent loop

agent loop 就住在 pi-ai 这层：模型指哪打哪，打完把结果递回去接着问



02 五层薄切：每层都薄得能一眼看完

ARCH

产出 **pi-mono** × 5 包

上下文纪律是底线：只有 AGENTS.md 被注入，系统提示词可整体替换

pi-coding-agent

CLI 组装层

会话 · 扩展 · 主题 · 上下文文件

三种模式：交互 TUI / print 单发 / headless (JSON 流 + RPC, 可当组件嵌进自己的程序)



事件流

pi-agent-core

Agent 运行时 (薄层)

Agent 类 · 状态 · 事件订阅 · 消息排队 · 传输抽象 —— "Everything is an event"



统一调用

pi-ai

统一多 provider LLM API + agent loop

Anthropic / OpenAI / Google / xAI / Groq / Cerebras / OpenRouter + GLM / Qwen / Kimi / DeepSeek / MiniMax

pi-tui

自研终端 UI: 差分渲染, 只重绘变化的部分

pi-telemetry

厂商中立遥测; 博客时代还没有, 是后来长出来的

<1000

启动 tokens; 省下的全留给代码

02 十个数字，勾出一个项目的轮廓

BY THE NUMBERS

抓取

2026-08-29

节奏像呼吸一样稳定：月均 5 个 release，两年发版 250 次

98.8k*

GITHUB STARS

250+

RELEASES 累计

v0.84.4

最新版 · 8 月 5 连发

5

NPM 包 · 单仓五层

<1000

启动 TOKENS

24

斜杠命令

50+

官方扩展示例

~30

API KEY PROVIDERS

6

刻意不做 · 每个都有替代路径

3

运行模式 · TUI / print / headless

WEAK * 星数为单源页面抓取（2026-08-29），未二次核验；「50+」「~30」为官网与 npm registry 口径。订阅登录另认 Claude Pro/Max、ChatGPT Codex、GitHub Copilot 三家。

03 四工具够用：模糊匹配是 edit 的机关

PHILOSOPHY

产出 read/write/edit/bash

模型自己的判断力是第一道防线——系统提示词全文不到 1000 tokens

read

分页 + 行号，按 offset 翻页，能读图片

write

整文件覆盖，没有花活

edit

diff 格式 + 模糊匹配：容忍空格缩进漂移，成功率就上来了

bash

执行命令 + 超时；长驻进程交给 tmux



You don't need to ask for permission. The user expects you to work autonomously until the task is complete. —— 遇到不认识的符号，先列出候选来源，挨个查证，把理解锚在可验证的代码或文档上。

SYSTEM PROMPT · **grep/find/ls** 只读工具默认关，**--tools** 想开就开

安全 · 姿态一

容器化

无内置权限系统，按启动用户权限跑；官方点名 Docker / Gondolin / OpenShell。

安全 · 姿态二

只读模式

pi --tools read,grep,find,ls —— 只带眼睛不带手，不靠弹窗靠裁剪。

安全 · 姿态三

供应链纪律

--ignore-scripts 跳过生命周期脚本 + 依赖锁版 + 最小存活期 + 白名单。

03 六个"No"是立场，不是缺口

PHILOSOPHY

产出

fig-3

每个 No 后面那句"你可以自己造"才是正主，第五章看官方交的作业

刻意不做	官方给的替代路径（README Philosophy 原文意译）
No MCP	用带 README 的 CLI 工具（见 Skills），或写扩展加上 MCP 支持
No sub-agents	用 tmux 起 pi 实例，或写扩展，或装别人做好的包
No permission popups	放容器里跑，或用扩展写自己的确认流程
No plan mode	计划写成文件，或写扩展，或装包
No built-in to-dos	原话是"它们会把模型搞糊涂"，用 TODO.md 或扩展
No background bash	用 tmux，全量可观测，直接交互

NOTE 第一遍读以为是偷懒的体面说法，第二遍才反应过来：这六个 No 是立场，不是缺口。

怎么读这张表

左列是"不做事"，右列全是"你自己动手的路径"——没有一条是"等官方做"。

怎么验证

本表为 README Philosophy 段原文逐字意译，可对照仓库复核。

04 十分钟跑起来，第一个坑在 Node 版本

HANDS-ON

产出

quick start

双轨登录：API key 按量付费走模型厂，/login 订阅走包月，pi 不抽成

```
quick start
```

```
# 安装 (Node ≥ 22.19.0, v0.84.4)
```

```
npm install -g --ignore-scripts \
  @earendil-works/pi-coding-agent
```

```
# 登录二选一
```

```
export ANTHROPIC_API_KEY=sk-ant- ...
pi
```

```
# 或交互里走订阅
/login
```

```
# 会话操作
```

```
pi -c          # 继续最近会话
```

```
pi -r          # 浏览历史会话
```

```
pi --fork id # 分叉新会话
```

坑 1

Node 版本

engines 写死 ≥22.19.0，不少默认 LTS 还停在 20，先 node -v。Windows 有 install.ps1 + 内置 powershell 工具，Termux 也有文档。

登录

双轨制

API key 约 30 家按量付费；/login 认三家订阅（Claude Pro/Max、ChatGPT Codex、Copilot）。本地模型：llama.cpp 一等公民 + Ollama。

账

两轨怎么选

API 贵时一晚烧几十块但模型随便挑；订阅额度内随使用。pi 对两轨功能零差异——它不关心你从哪儿付钱。

04 会话是一棵树：跳分支不改历史，只换路径

HANDS-ON

产出 `/tree`

Esc × 2 进树视图是最有辨识度的操作；Enter 与 Alt+Enter 是两种排队语义

STEP 1

安装

`npm -g · Node ≥ 22.19`



STEP 2

登录

API key 或 `/login` 订阅



STEP 3

首会话

pi 敲下第一句



会话树

Esc × 2 进树视图

树怎么存

JSONL + id/parentId

消息、工具调用、压缩摘要都是节点；当前所在位置叫 active leaf。跳分支不改历史，被放弃的分支自动做摘要挂在岔路口。

怎么跳

/fork vs /clone

`/fork`：从历史消息开新会话，"回到过去改个决定"；`/clone`：当前分支整个复制，"平行世界再来一遍"。

怎么排队

Enter vs Alt+Enter

Enter 是 steering，本轮工具跑完就插入，用来纠偏；Alt+Enter 是 follow-up，全部干完才送，用来喂任务。

KEYS · 迁移 Ctrl+L 换模型 · Ctrl+P 轮换 · Shift+Tab 思考档 · `/compact` 有 · CLAUDE.md 也认——Claude Code 用户肌肉记忆大体能用，24 个命令 `/hotkeys` 查全。

05 被拒掉的能力，扩展系统里全能造回来

EXTENSIONS

产出

ExtensionAPI

API 就三件事：`registerTool` · `registerCommand` · `on("tool_call")`

Extensions

TS 代码：工具 · 命令 · 事件 · 快捷键 · TUI

Skills

agentskills.io 标准，`/skill:name` 调用



Prompt Templates

/模板名展开，`{{变量}}` 占位

Themes

整套换皮



自造能力

`sub-agent` · `plan mode` · 权限门 · 沙箱 · 自定义压缩

pi package

`pi install npm:/git`: 一条命令分发

API 三件套

能拦截每一次工具调用

扩展 = 默认导出的工厂函数，放
`~/.pi/agent/extensions/` 或
`.pi/extensions/`，`/reload` 立即生效。`tool_call` 事件
返回 `{ block: true }` 即可阻止。

官方作业

50+ 扩展示例

`subagent/`、`plan-mode/`、`permission-gate`、
`sandbox/`、等结果打 Doom 的扩展——六个 "No" 每个
都有参考实现。社区有 `oh-my-pi` (LSP+浏览器+子代理)、`pi-mom` (Slack)。

提醒

先读源码再装

`pi` 包以完整系统权限运行（官方原话）。作者把自己的开发会话全公开在 Hugging Face——这个项目是被它自己造出来的。

05 半分钟读完你 agent 的全部行为

EXTENSIONS

产出 `protected-paths.ts`

拦截 `tool_call` 事件，命中保护路径就返回 `block`

```
packages/coding-agent/examples/extensions/protected-paths.ts
import type { ExtensionAPI } from "@earendil-works/pi-coding-agent";

export default function (pi: ExtensionAPI) {
  const protectedPaths = [".env", ".git/", "node_modules/"];

  pi.on("tool_call", async (event, ctx) => {
    if (event.toolName !== "write" && event.toolName !== "edit") {
      return undefined;
    }
    const path = event.input.path as string;
    const isProtected = protectedPaths.some((p) => path.includes(p));
    if (isProtected) {
      if (ctx.hasUI) {
        ctx.ui.notify(`Blocked write to protected path: ${path}`, "warning");
      }
      return { block: true, reason: `Path "${path}" is protected` };
    }
    return undefined;
  });
}
```

READ

三件套

`registerTool` 注册工具 · `registerCommand` 注册命令 · `on("tool_call")` 拦截事件，返回 `{ block: true, reason }` 即可阻止。

ECO

社区已经造好的

`oh-my-pi` (hash 锚定编辑+LSP+浏览器+子代理)、`pi-mom` (Slack 委派)、`@termdraw/pi` (画图)、`pi-doom` (等结果时打 Doom)。

NOTE 半分钟读完 = 你真的可以拥有你的 agent 的全部行为。

06 分歧点只有一个：能力边界由谁决定

COMPARE

产出

fig-6

Claude Code 和 OpenCode 把 MCP 当标配，pi 刻意不做——这是分水岭

维度	pi	Claude Code	OpenCode
定位	极简 agent harness	官方全能 coding 工具	开源多形态 agent
许可	MIT，全开源	闭源订阅制（通识，存疑）	开源（Anomaly）
MCP	刻意不做，可扩展补	有	有
sub-agent	无内置，有示例扩展	有	有
扩展模型	TS 扩展 + pi package	MCP + hooks + skills	MCP + plugins + SDK
计费	免费，无定价页	订阅制	未标注

| 空缺即诚实

表格空着的格子是未取得一手数据的地方，宁有空着不编。

| 完整九维

内置工具 / 模型 / 形态三行对照见完整版文章第 06 章。

06 选底盘还是选整车，取决于你想不想要所有权

COMPARE

补充

模型自由度

没有对错，只有你想要什么

底盘：pi

每一处都能改：扩展、skills、主题全是你的代码；半分钟读完一个扩展的全部行为。

代价：组装和维护归你——大仓检索靠 grep 加耐心，并行靠 tmux 自排，自建扩展随模型升级自己修。

整车：Claude Code / OpenCode

上路就能开：MCP、sub-agent、权限确认、plan mode、后台任务全是产品功能，IDE 插件和桌面端齐备。

代价：路线是别人画的，能力边界由厂商决定，模型选择受订阅框架约束。

BONUS pi 的 /model 会话内热切换 + 跨 provider 上下文接力：上半场 Claude 读库、下半场 DeepSeek 干活，一条 workflow 混用两家模型是常规操作。

07 极简不是免费的：六笔账每笔都有出处

CRITIQUE

产出

六笔账

前四笔是"少"的代价，后两笔是"火"的代价

BILL 01

无 MCP

头号拥趸都列为第一缺点；2026 年 MCP 已是事实标准，现成服务器全接不了。

BILL 02

无现成生态

"no community skills, nothing"——能力获取基本等于自己写代码。

BILL 03

TS 门槛

深度定制的入场券是会写 TypeScript（推断，无用户引语）。

BILL 04

自建维护成本

自建扩展是自己的代码，模型升级行为漂移了得自己修。

BILL 05

商业化悬念

"enshitification?"质疑 vs MIT 承诺 + Fair Source 分层；迁移成本已真实发生。

BILL 06

规模边界

大仓检索靠 grep 加耐心，多任务并行靠 tmux——编排复杂度回到你手上。

HONEST Earendil 靠什么把 pi 变成生意，我没查明白。pi.dev 上没有定价页，这是我能确认的全部。

07 三段原声：引语逐字，立场标注

CRITIQUE

立场

已披露

好评圈层的账也要算：最重要的几篇好评，作者要么在 Earendil，要么和它互相依存

”

The most obvious omission is support for MCP. There is no MCP support in it.

– Armin Ronacher (Flask 作者) · 立场：自称 **shill**，与 **Zechner** 同在 **Earendil** · 自己人说“缺失最明显”，分量反而更重

”

pi is MIT licensed. It will stay MIT licensed. Nothing changes. —— 但未来商业功能会走 Fair Source。

– Mario Zechner, 官宣博文 · 背景：**r/LocalLLaMA** 立帖“**enshitification?**” · 截至发稿无背刺实锤，风险是结构性的

”

Or not, I don't know :)

– Ronacher 谈 MCP 桥接方案的可靠性 · 拥趸对核心替代路径的态度是“我不知道” · 另：HN 广传的“**linear flow** 沮丧”是在骂别家，不是骂 **pi**

07 五种情况别用 pi, 一种情况闭眼入

CRITIQUE

产出 fig-7

反方视角的适用边界，每一条都有出处

依赖 MCP 服务器生态

接工具要写 TS 扩展或桥接，而桥接可靠性连拥趸都承认"我不知道"。(Ronacher 原话)

不会 TS 又想深度定制

定制的入场券是会写 TypeScript; "重写 internals" 的另一面就是门槛。(推断，非用户引语)

对许可分层过敏的组织

核心 MIT 承诺在，但 Fair Source 商业层已写入路线，预期分化。(RFC 0015 转述)

要开箱即用的全家桶

sub-agent、内置 git、find、社区 skills 都没有，且这是设计立场不是功能路线图。(Reddit 实测 + "nothing")

IDE / GUI 重度用户

纯终端 harness，无官方图形前端；线性对话流是刻意选定的交互。

反过来，谁适合

想完全掌控 harness、愿意读源码、用 TypeScript 的高级开发者与 agent 构建者。

提炼：pi 的价值不在"更好用"，在"可拥有"

SUMMARY

pi · A MINIMAL AGENT HARNESS

你能读完它的全部行为，改掉其中任何一处，然后真正拥有你的 agent

01	缘起是什么 对"塞满上下文、黑盒操作"投反对票，顺手被 OpenClaw 点爆	L26/L21
02	架构巧在哪 五层薄切，agent loop 住在模型接入层里，启动 <1000 tokens	L1/L2/L7
03	四工具的底气 覆盖 coding 全部动作原语，六个"No"各有替代路径，安全挪到环境侧	L3/L4/L5
04	上手难不难 一条 npm 命令，双轨登录，会话树和消息排队值得练熟	L10-L12
05	扩展的真相 "没有的"官方示例全能造，但示例和产品之间隔着你自己写的代码	L13-L15
06	账单有哪些 无 MCP、无生态、TS 门槛、自建维护、商业化悬念、好评圈层	L22-L24

想开现成的车，另外两家更合适；想拥有一台完全按自己想法改装的，pi 值得这场折腾。完整论述（含九维对照表与全部引文）见完整版 final.md 第 06-07 章。

weak 来源已在正文标注口径

pi 官网与文档 — pi.dev

仓库与 README (Philosophy / Quick Start / Extensions / Sessions) — github.com/earendil-works/pi

npm 包元数据 (engines / 版本) — registry.npmjs.org/@earendil-works/pi-coding-agent/latest

Zechner 设计博文 (2025-11-30) — mariozechner.at/posts/2025-11-30-pi-coding-agent

Zechner 《I've sold out》(2026-04-08) — mariozechner.at/posts/2026-04-08-ive-sold-out

Ronacher 评述 (2026-01-31, 立场见正文) — lucumr.pocoo.org/2026/1/31/pi/

protected-paths.ts 扩展源码 — github.com/earendil-works/pi/blob/main/packages/coding-agent/examples/extensions

会话文档 — […/packages/coding-agent/docs/sessions.md](https://github.com/earendil-works/pi/blob/main/packages/coding-agent/docs/sessions.md)

GitHub Releases (v0.84.4, 2026-08-28) — github.com/earendil-works/pi/releases

Reddit 实测评 (转引) — reddit.com/r/AI_Agent_Reviews/comments/1t62nnj

r/LocalLLaMA 商业化质疑帖 (转引) — reddit.com/r/LocalLLaMA/comments/1u6a499

对比数据 (Claude Code / OpenCode) — code.claude.com/docs/en/overview · opencode.ai/docs (空缺处为未取得一手数据)

WEAK GitHub 星数 98.8k 为单源页面抓取 (2026-08-29); oh-my-pi 等社区包描述来自官网与二手综述, 未逐仓核验; Claude Code 闭源为业内通识, 已按存疑处理。

MAP 降级地图: 本 PPT 为压缩版——九维对照全表在完整版 § 06; 三段原声的完整上下文在 § 07; 26 条调研台账在 research/ledger.md; PPT 删减的叙述性论证均在 final.md 同名章节。